

Agenda

- Interfaces
- Abstract classes
- Static
- Final

INTERFACES



→ Blueprint of an entity

→ attributes

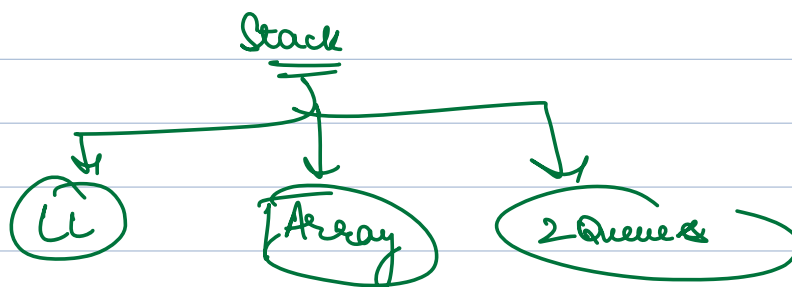
→ behaviour.

→ Sometimes an entity is characterized by the behaviours it supports.

Defined
by

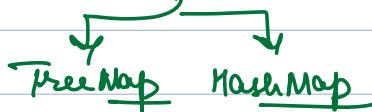
Stack → AIFO → Push, Pop, top()

Queue → LIFO → enqueue() dequeue



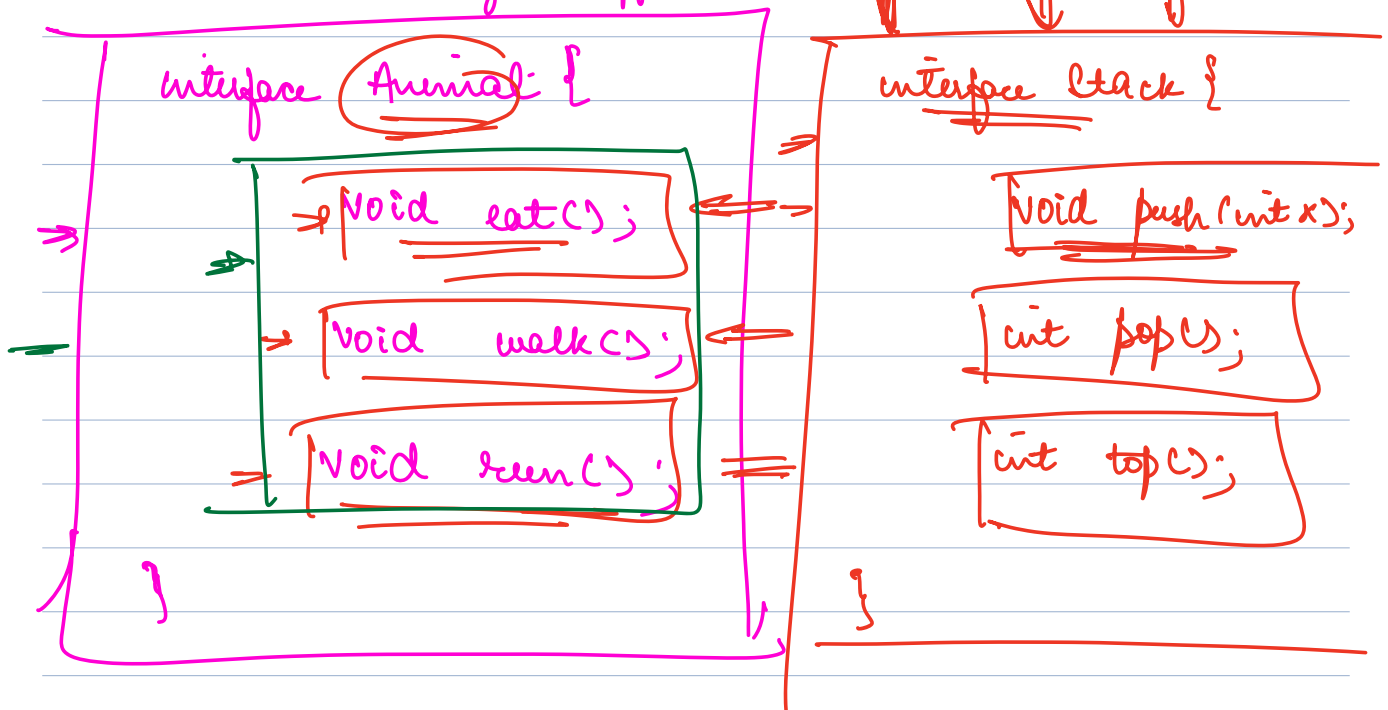
→ You don't care about how that behaviour is implemented and only care about what all behaviours are supported.

Eg: Stack, Queue, Map (K, V) put() get()



Interfaces: Blueprint of a behaviour =

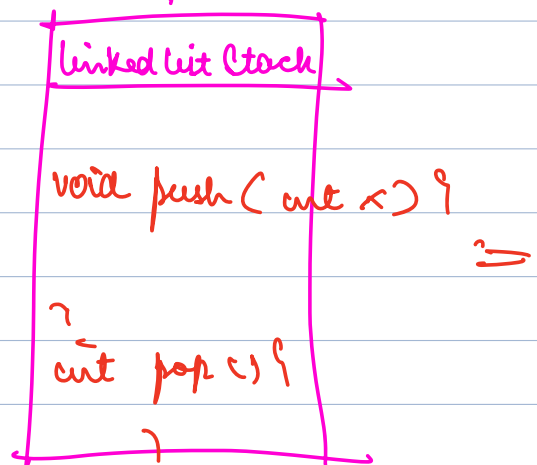
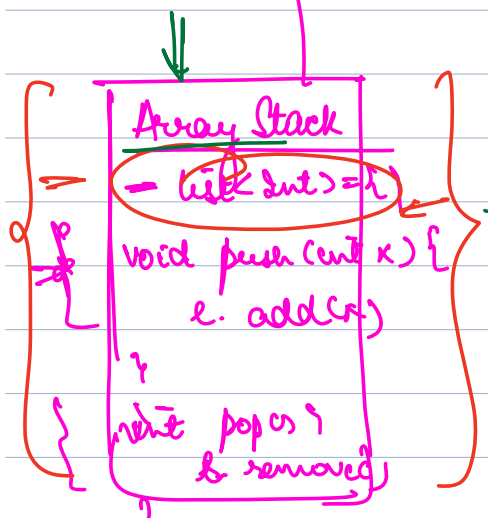
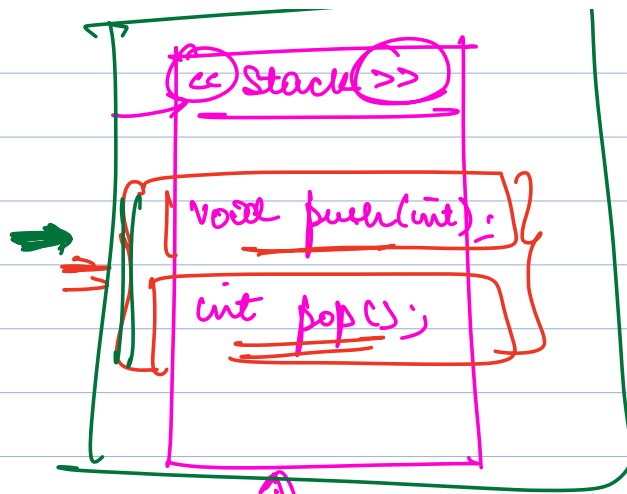
⇒ a way to define entities that are characterized by the behaviours they support.



⇒ Attributes are often considered unimportant details.

Interface == Contract

⇒ a class has to implement an interface.



implementing an interface == satisfying the contract

class ArrayStack implements Stack {

→ [private List<Integer> array = new ArrayList<>();

public void push (int x) {
addToStack(x);
}

→ [private void addToStack (int x) {
array.add(x);
}

}

→ Why interface if no imp? → POLYMORPHISM

→ Code maintainable.

→ Code to interface not to an implementation

class

interface List l = new Array List (> C) ← class.

```

class Client {
    private Stack stack;
    // new Array Stack()
    // new linked list Stack()
    stack.push();
    stack.pop();
}

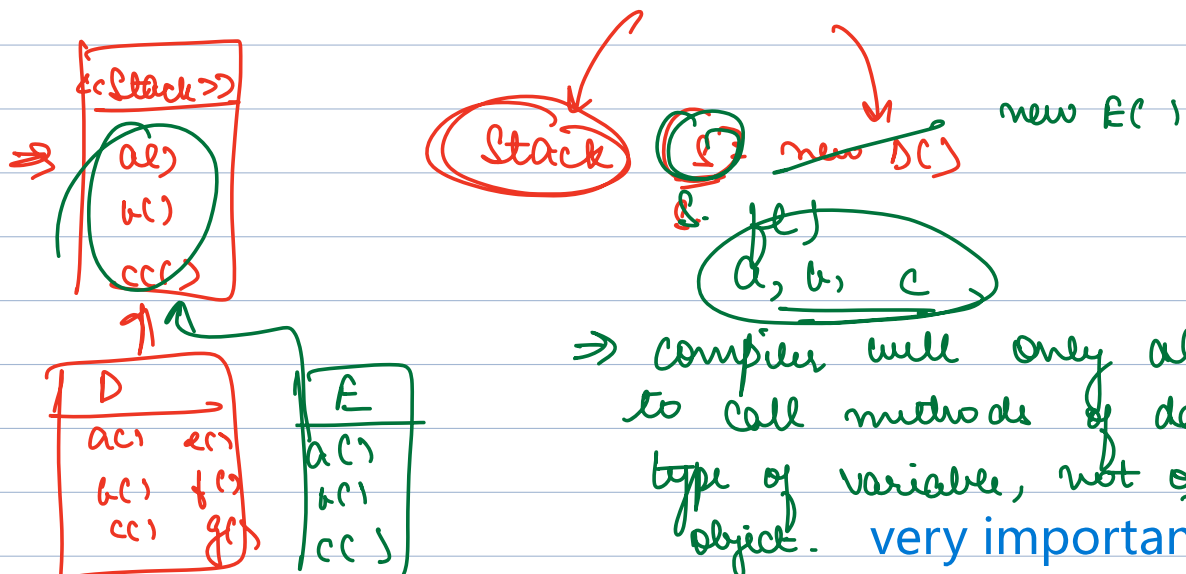
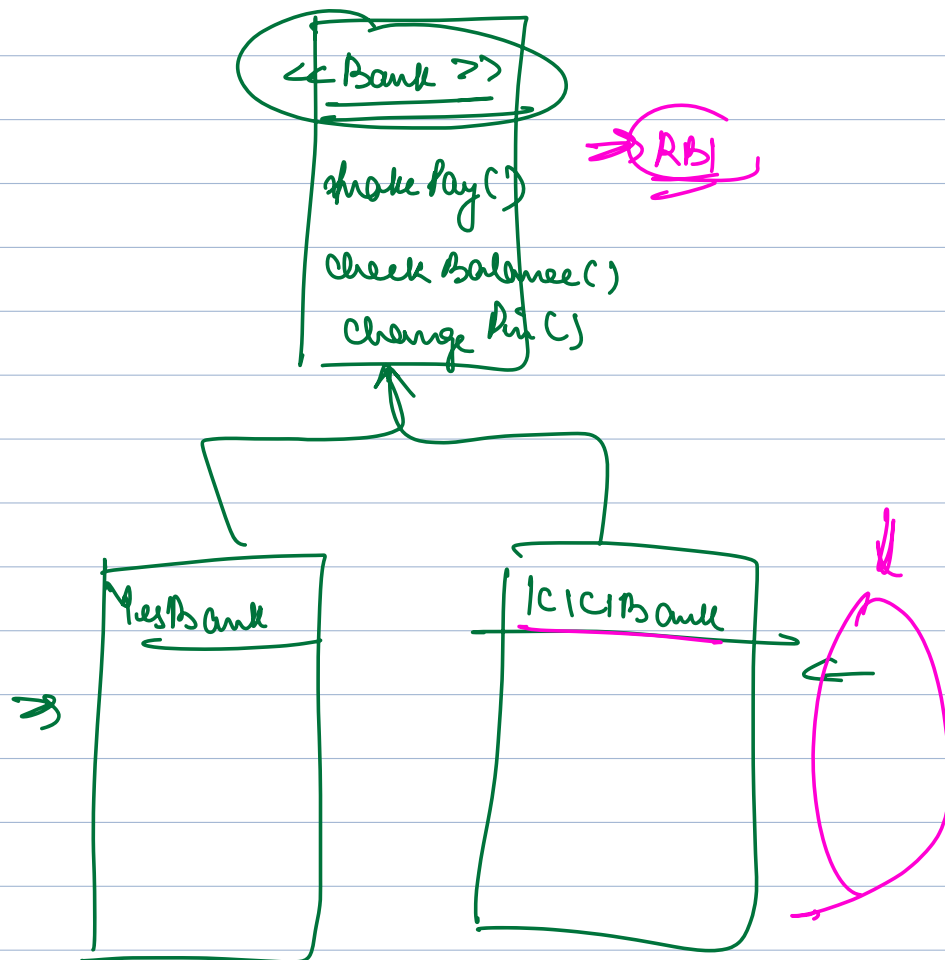
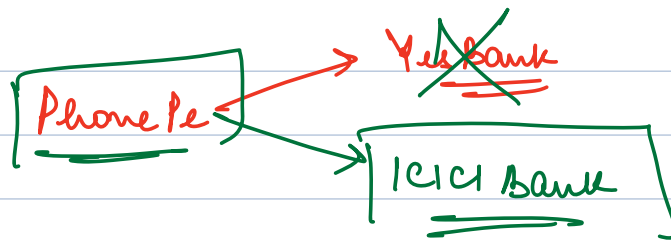
```

Stack s = ~~new Stack()~~

→ Can't create
an obj of
interface

Stack s = new Array Stack()

= new linked list Stack()



→ compiler will only allow to call methods of data type of variable, not of object. **very important**

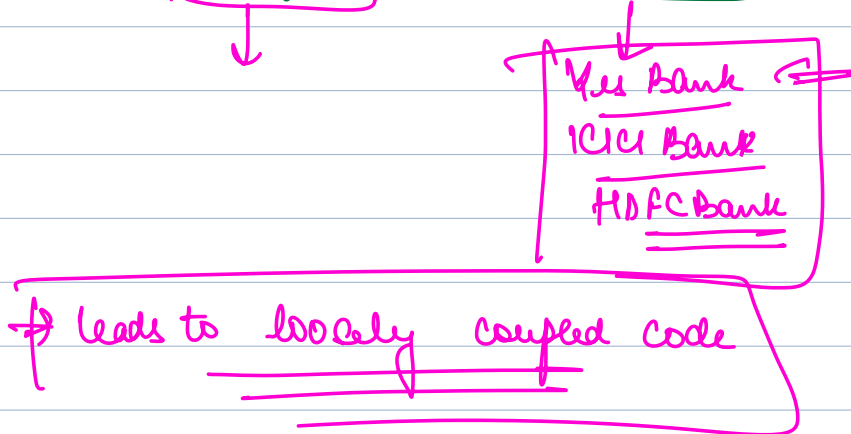
```

S getStack() {
    int i = new RandC()
    if (i == 1)
        new DC()
    else new E()
}

Stack s = getStack()

```

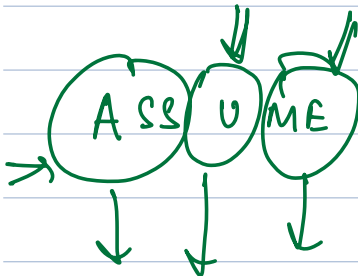
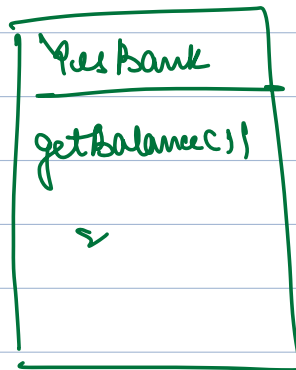
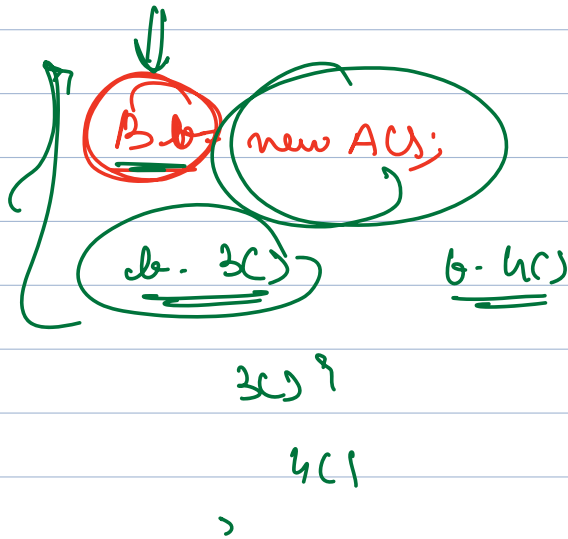
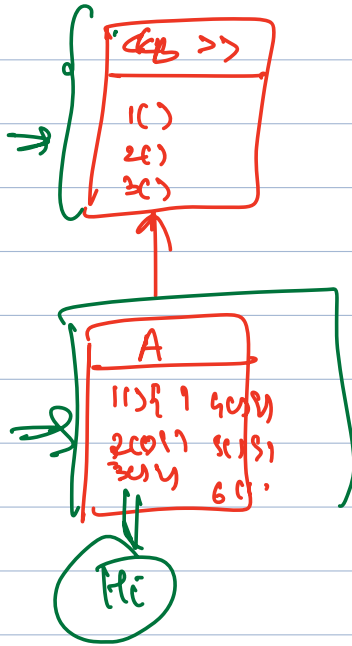
⇒ Code to interface not an imp



Interface v/s Class

- ① Can't create an obj for an interface
- ② No implementation in an interface.

OOP: interface

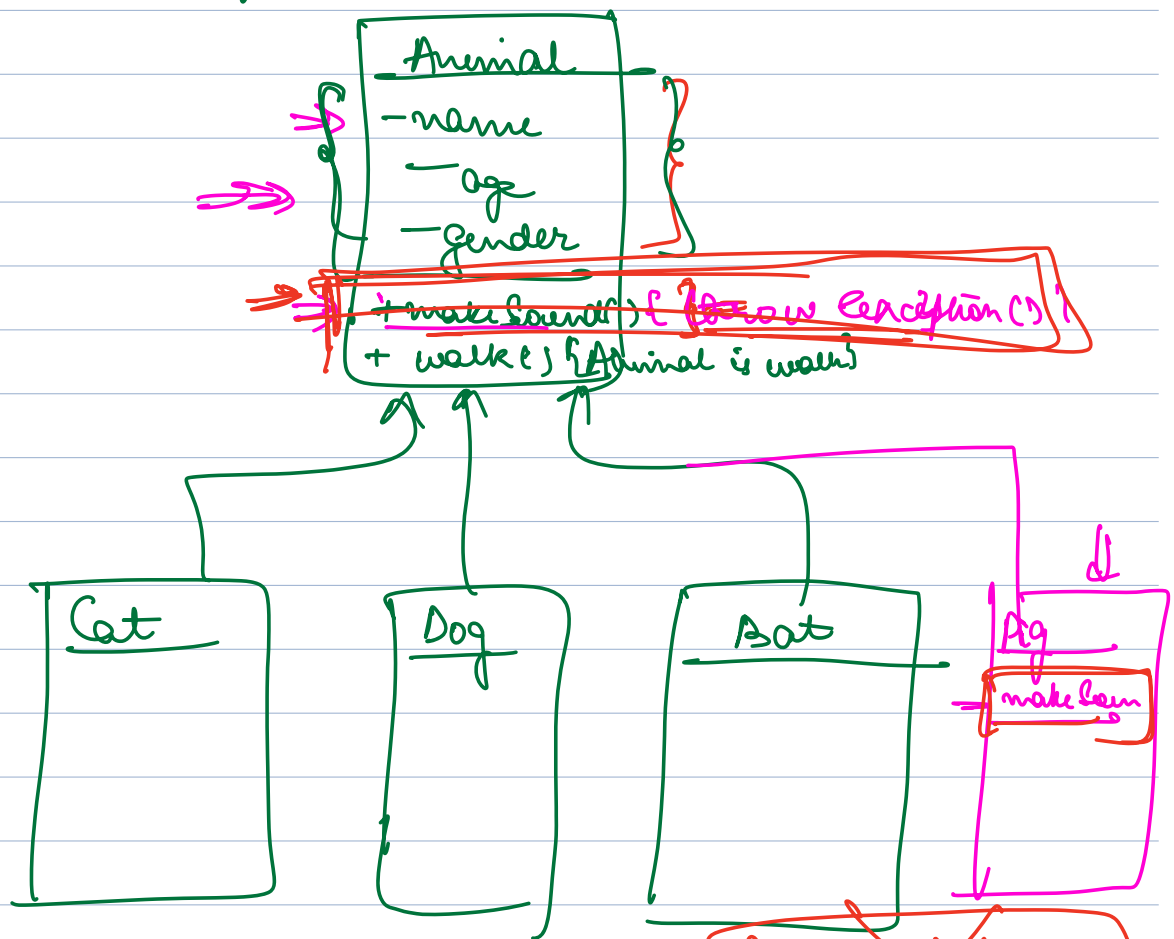


ABSTRACT

CLASSES

entity

→ Sometimes when we are defining entities with a hierarchy (inheritance) we may not be able to properly define all the methods of parent class.



Options for make Sound:

- ① leave empty → a child class may forget to implement that.
- ② interface → no attrs

~~Pig p = new Pig()
p.makeSound()~~

⇒ We need a way to define an entity where all methods may not be defined and compiler must ensure that the child classes define that method.

↓
Abstract classes
↓

way to define an entity w/o defining all methods.

```
Abstract class Animal {  
    private String name;  
    private int age;  
    private String gender();  
  
    public void eat() {  
        cout << "Animal is eating";  
    }  
  
    public abstract void makeSound();  
}
```

class Dog extends Animal {

```
public void makeSound() {  
    cout << "Bhow" ;  
}
```

~~Animal a = new Animal();~~
⇒ impl is not complete

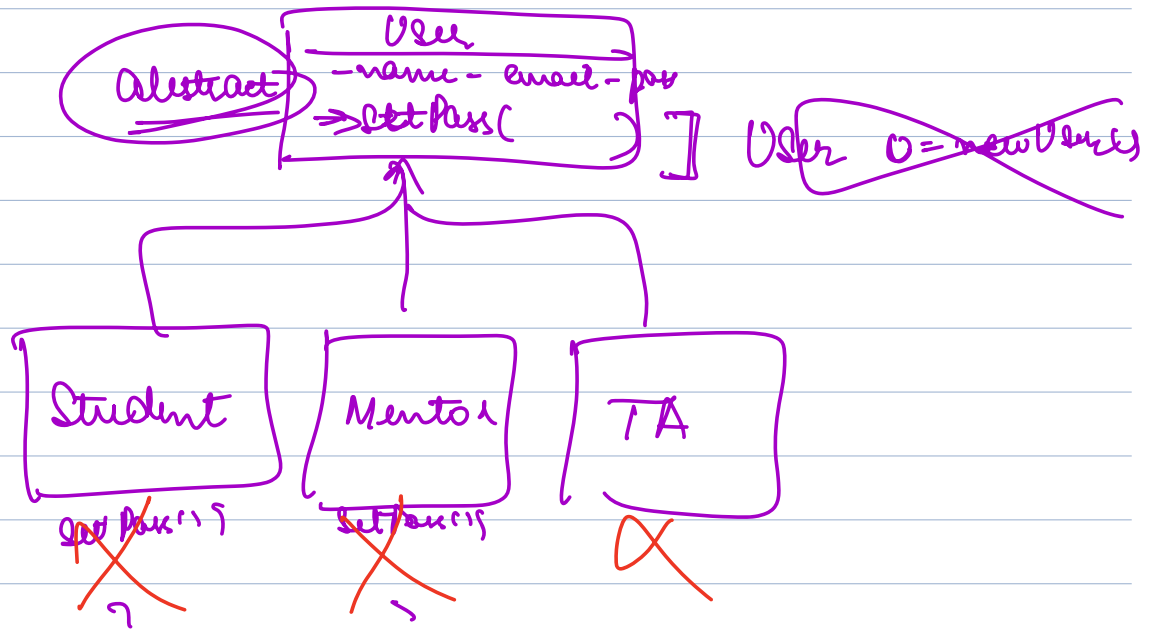
→ Child class must implement abstract method
(OK) itself be declared abstract

```
abstract class Dog extends Animal {}
```

```
class Shitzu extends Dog {  
    makeSound() {  
        ==  
    }  
}
```

⇒ A class can be declared abs even if it doesn't have any abs method.

⇒ because you might not want to create obj of it



Break till 10:30 PM

→ abs class ✓

→ access modifiers and inheritance ✓

→ attr/ in Java interface ✓

→ static defined methods

→ final

→ Diamond Problem ✓

- ① Define a parent class w/o imp of some methods
- ② Define a class whose object you don't want to create

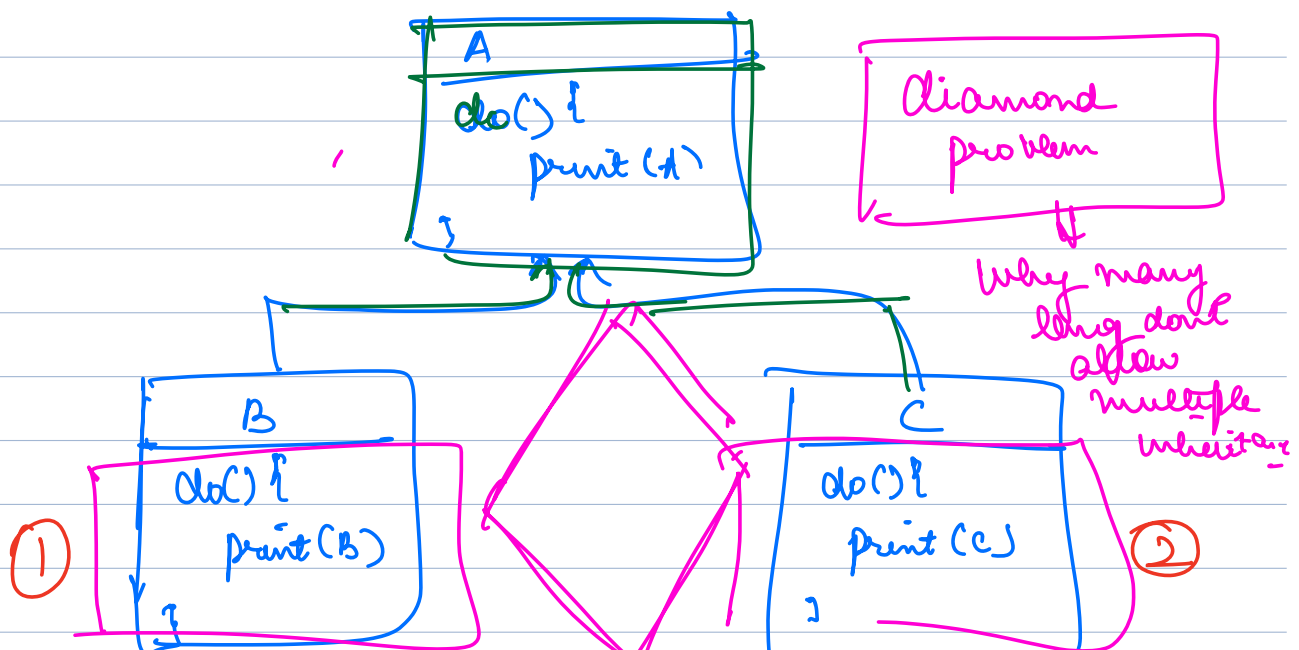
Abstr Class v/s Interface

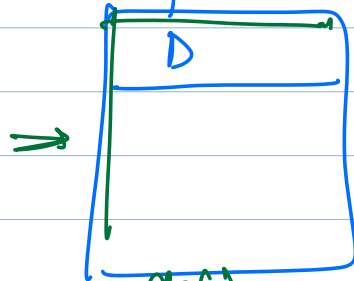
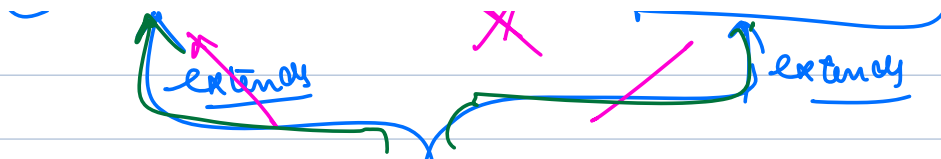
① Abstr class $\Rightarrow$ entity (attr) methods)
 $\Rightarrow$ Interface $\Rightarrow$ contract that others should follow

② You can't create an obj of any of these.

③ One class can extend only 1 other class
 $\Rightarrow$ whereas one class can implement multiple interfaces.

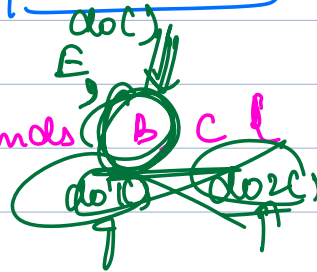
Why extend only one other class $\Rightarrow$ Diamond Problem.





D d = new D();
d.do();

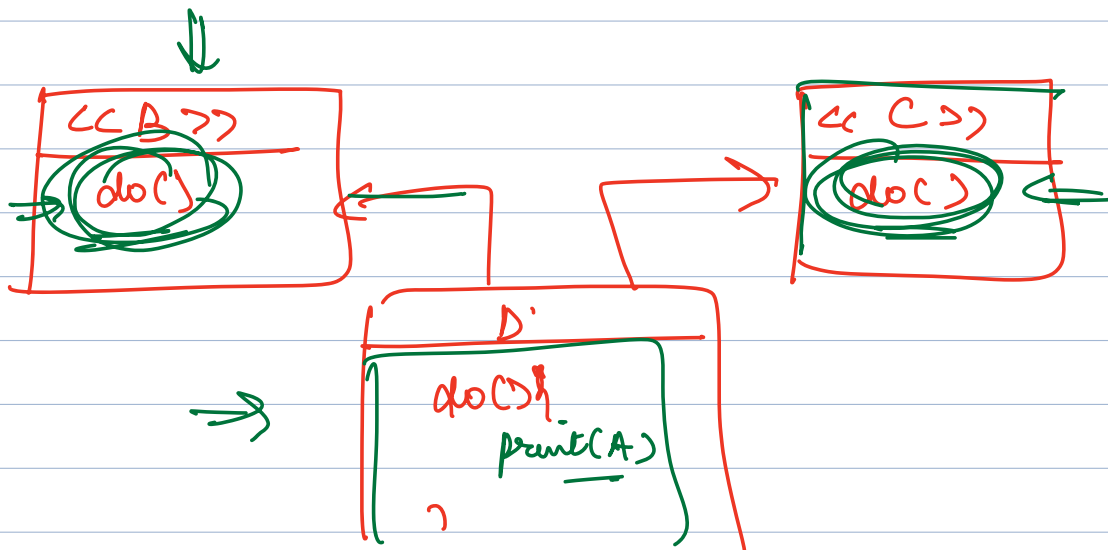
Class A extends

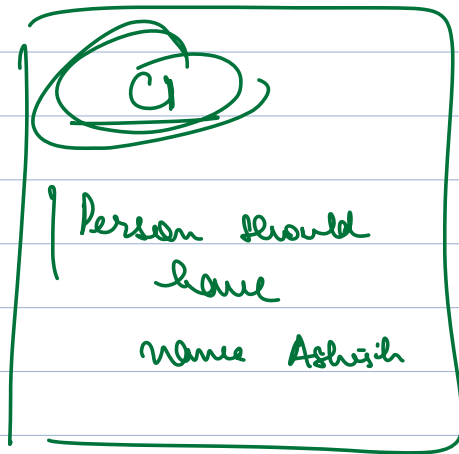


A a = new A();

a.do();

in case of ambiguity
 call of first class

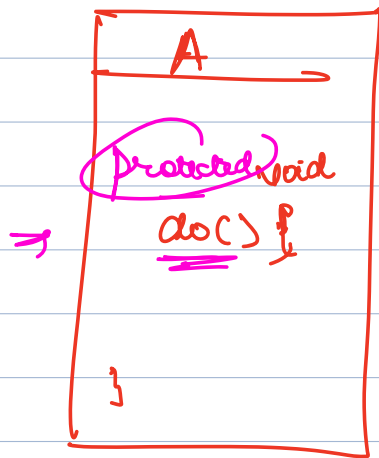




(4) Always prefer interfaces

Only create abs class if:

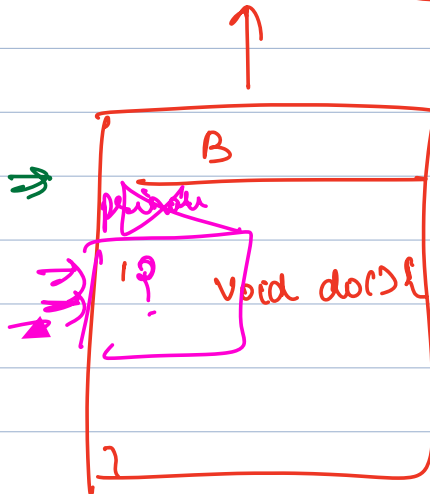
- ① you need to create a class up to allowing its obj to be created
- ② in an inheritance relⁿ you are not able to define all ~~obj~~ methods in a parent class



~~public~~

① private

② public, default



⇒ Can a person access ~~variable~~ ^{obj} of child class with variable of parent

⇒ A a = new B()

⇒ Allowed to call members of A

based on access modifier of A

⇒ a.do()

or keep same

⇒ child class can only broaden the access / it can't reduce the scope.

Child $\geq$ parent

↓

Java 8 $\Rightarrow$ Java allowed default imp in interfaces as well as attr in interfaces.

$\Rightarrow$ Can Java interfaces have attr $\Rightarrow$ YES
But they are not object level
 $\Rightarrow$ They are nothing but constants.

interface APIEndpoints {

String url = "/v1/api/user/"

String mentor = _____

$\Rightarrow$ interfaces can have attr $\Rightarrow$ constants
in Java a constant can be defined
via final

public final

$\rightarrow$ Can only assign value to a final variable once

`private final List<Integer> l = {1, 2, 3, 4};`

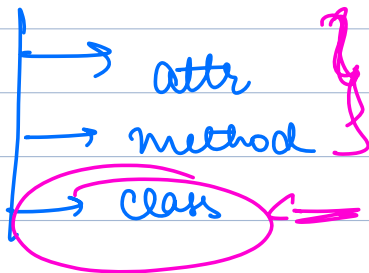
b. `add(6)`

c. `remove()`

`l = new ArrayList()`

Value of variable `l` final Obj in the variable may change

Static



Builder design pattern

⇒ used to define members that are class level and not object level.

`class Animal {`
 `public static int count = 0;`

Associate to class.

}

Animal a = new Animal()

Animal b = new Animal()

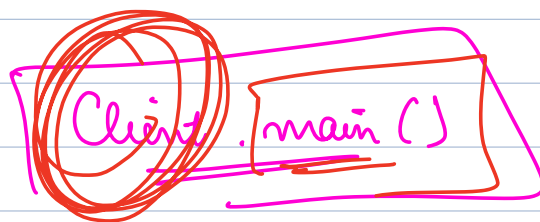
Animal.count += 1

class Client {

psym() {

}

}



⇒ when you start
prog there is no
obj.

